📚 DAY 2: MASTER TERMINOLOGY & CONCEPT GLOSSARY

High Performance Traditional Machine Learning & Parallel Computing using Intel oneAPI

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 2 (Strictly 3 Hours / 180 Minutes)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

📌 Master Vocabulary & Silicon Execution Mechanics (Terms #1 – #36)

1. Amdahl's Law

- **Definition**: A mathematical formulation that determines the theoretical maximum speedup limit of an application when using N parallel processors.
- **Equation**: $\text{Speedup}(N) = \frac{1}{(1-P) + \frac{P}{N}}$, where P is the parallelizable fraction of the code, and $(1 - P)$ is the strictly serial fraction.
- **HPC Silicon Mechanism**: Highlights the "serial bottleneck". Even if you allocate 10,000 Xeon CPU cores, if 5% of your code is serial ($P = 0.95$), the theoretical maximum speedup can never exceed $\frac{1}{0.05} = 20\times$.
- **Trainer Script**: "*Students, Amdahl's law teaches us that adding more hardware cannot fix bad serial code. You must minimize serial bottlenecks before throwing 100 cores at a problem.*"

2. Gustafson-Barsis Law

- **Definition**: A law of scaled speedup stating that as computational resources increase, the problem size itself expands to fill the available memory.
- **Equation**: $\text{Speedup}(N) = N - \alpha(N - 1)$, where α is the serial fraction.
- **HPC Silicon Mechanism**: Assumes workload scales with the machine size, proving that parallel efficiency remains high when computing massive multi-gigabyte datasets across large clusters.

3. Task Parallelism vs. Data Parallelism

- **Definition**:
  - **Data Parallelism**: The same computational operation is executed simultaneously across different subsets of data (e.g., matrix addition, batch gradient descent).
  - **Task Parallelism**: Completely distinct operations/functions run concurrently on different threads (e.g., data ingestion thread vs. computational worker thread vs. I/O logger).
- **HPC Silicon Mechanism**: Data parallelism maps directly to SIMD vector registers and GPU streaming multiprocessors; task parallelism maps to CPU thread pools managed by oneTBB or OpenMP sections.

4. Fork-Join Model (OpenMP)

- **Definition**: The fundamental execution paradigm of OpenMP. An application begins as a single master thread (serial execution). When encountering a parallel region, the master thread **forks** a team of worker threads across CPU cores. Upon completion, threads synchronize at an implicit barrier and **join** back into the master thread.
- **HPC Silicon Mechanism**: Worker threads are pinned to physical CPU cores via `GOMP_CPU_AFFINITY` or `KMP_AFFINITY=granularity=fine,compact,1,0` to prevent operating system core hopping and cache invalidation.

5. OpenMP Directives (`#pragma omp`)

- **Definition**: Compiler pragmas in C/C++ that instruct the compiler to generate multithreaded parallel runtime code without requiring low-level POSIX pthread management.
- **Key Directives**:
  - `#pragma omp parallel`: Spawns a team of threads.
  - `#pragma omp for`: Distributes loop iterations across the active thread team.
  - `#pragma omp single`: Directs only one thread to execute the structured block.
  - `#pragma omp barrier`: Enforces explicit hardware synchronization.

6. Reduction Clause (`reduction(operator : variable)`)

- **Definition**: An OpenMP clause that prevents race conditions during associative arithmetic operations (e.g., summation, dot product, min/max).
- **HPC Silicon Mechanism**: Each thread allocates a private accumulator variable residing in its local L1 cache or CPU register. During the loop, threads update their local accumulators with zero lock overhead. At loop termination, OpenMP combines all private values into the shared variable using a tree reduction.

7. Race Condition & Critical Section

- **Definition**: A concurrency flaw where two or more threads attempt to read and write to the same shared memory location simultaneously, leading to non-deterministic, corrupted results.
- **Fix Mechanisms**:
  - `#pragma omp critical`: Allows only one thread at a time to execute the code block (high locking overhead).
  - `#pragma omp atomic`: Hardware-enforced atomic CPU instruction (e.g., `LOCK XADD` on x86), orders of magnitude faster than a mutex.

8. False Sharing (The Silent Cache Destroyer)

- **Definition**: A severe performance degradation that occurs when two independent CPU cores concurrently write to distinct variables that reside within the **SAME 64-byte L1 CPU cache line**.
- **HPC Silicon Mechanism**: Under the hardware MESI (Modified, Exclusive, Shared, Invalid) cache coherence protocol, Core 0 writing to variable `A` forces the CPU bus to send an invalidation signal to Core 1's cache containing variable `B`, causing continuous cache misses and bouncing latency from 1 ns to 80 ns.
- **Remedy**: Pad variables to 64-byte boundaries using `alignas(64)` or use private loop variables.

9. 1D & 2D Domain Decomposition

- **Definition**: Dividing a large spatial grid, array, or dataset into smaller contiguous geometric sub-blocks distributed across processors.
- **HPC Silicon Mechanism**: Keeps data contiguous in RAM, maximizing L1/L2 cache hit rates and enabling SIMD hardware prefetchers to stream data ahead of execution.

10. Halo Exchange / Ghost Cells

- **Definition**: Boundary data cells exchanged between adjacent parallel sub-domains during distributed stencil computations (e.g., fluid dynamics, image filtering).
- **HPC Silicon Mechanism**: Handled via asynchronous non-blocking MPI calls (`MPI_Isend` / `MPI_Irecv`) to overlap network transfer with internal grid computation.

11. Distributed Memory Architecture

- **Definition**: A cluster architecture where each compute node has its own private physical RAM and operating system. Processors cannot directly read or write to RAM on other nodes; all sharing requires explicit network messaging.

12. MPI Communicator & `MPI_COMM_WORLD`

- **Definition**: An opaque identifier representing a group of processes that can exchange messages.
- `MPI_COMM_WORLD` : The default communicator created at `MPI_Init()` , encompassing all processes spawned by `mpirun` .

13. MPI Process Rank & Size

- **Definition**:
  - **Rank (`MPI_Comm_rank`)**: The unique integer identifier assigned to each process inside a communicator (from 0 to $\text{Size} - 1$).
  - **Size (`MPI_Comm_size`)**: The total number of processes participating in the communicator.

14. Point-to-Point Communication (`MPI_Send` / `MPI_Recv`)

- **Definition**: Direct message transfer between exactly two processes: one sender rank and one receiver rank.
- **Syntax**: `MPI_Send(buffer, count, datatype, dest_rank, tag, comm)` paired with `MPI_Recv(...)` .

15. Collective Communication: Broadcast (`MPI_Bcast`)

- **Definition**: One root process transmits an identical data buffer to all processes within the communicator simultaneously.
- **HPC Silicon Mechanism**: Uses an optimized binary tree or binomial graph topology across the network switch to achieve $O(\log N)$ latency instead of sequential $O(N)$ copies.

16. Collective Communication: Scatter & Gather

- **Definition**:
  - `MPI_Scatter` : Takes an array on the root rank, splits it into equal segments, and sends one segment to each process.
  - `MPI_Gather` : Reverses scatter; collects individual results from all processes and concatenates them into a unified array on the root.

17. Collective Communication: Reduction (`MPI_Reduce` & `MPI_Allreduce`)

- **Definition**: Collects data from all ranks, performs an associative reduction operation (e.g., `MPI_SUM` , `MPI_MAX` , `MPI_MIN`), and delivers the final scalar/vector to the root (`MPI_Reduce`) or to every rank (`MPI_Allreduce`).

18. Intel AI Analytics Toolkit (AI Kit)

- **Definition**: A specialized software bundle within the Intel oneAPI ecosystem containing optimized Python runtimes, machine learning libraries, and deep learning frameworks engineered to deliver maximum throughput on Intel Xeon CPUs and GPUs.
- **Key Components**: Intel Distribution for Python, Intel-optimized NumPy, Intel-optimized Pandas, Intel Extension for Scikit-learn (`sklearnex`), Intel Extension for PyTorch (`IPEX`), and Intel-optimized TensorFlow.

19. Intel oneDAL (oneAPI Data Analytics Library)

- **Definition**: A C++ and SYCL data analytics library providing highly tuned algorithmic building blocks for all stages of data analysis: preprocessing, transformation, clustering, regression, classification, and model evaluation.
- **Heritage**: Formerly known as Intel DAAL (Data Analytics Acceleration Library).

20. Intel Extension for Scikit-learn (`sklearnex`)

- **Definition**: A seamless acceleration package that dynamically patches standard Python Scikit-learn algorithms with Intel oneDAL C++ implementations.
- **Code Implementation**:

```
from sklearnex import patch_sklearn
patch_sklearn()
```

- **Performance Gain**: Delivers 10× to 100× speedups on multi-core Xeon CPUs with **zero changes to user model code**.

21. Supported `sklearnex` Algorithms

- **Clustering**: K-Means, DBSCAN.
- **Dimensionality Reduction**: Principal Component Analysis (PCA), t-SNE.
- **Classification**: Support Vector Machines (SVC), Random Forest Classifier, K-Nearest Neighbors (k-NN), Logistic Regression.
- **Regression**: Random Forest Regressor, Linear Regression, Ridge Regression, ElasticNet.

22. Cache-Conscious GEMM & BLAS Level 3

- **Definition**: Matrix-Matrix multiplication operations ($C = \alpha AB + \beta C$) partitioned into cache-sized sub-matrices (cache blocking) so operands fit entirely in L1/L2 caches, eliminating main memory stalls.
- **HPC Silicon Mechanism**: oneDAL and oneMKL divide matrices into tiles matching Xeon's 1.25 MB L2 cache and invoke AVX-512 FMA (Fused Multiply-Add) instructions.

23. Intel-Optimized NumPy (Backed by oneMKL)

- **Definition**: A drop-in replacement for standard NumPy where linear algebra routines (`np.dot`, `np.linalg.svd`, `np.linalg.qr`, `np.fft`) link directly to Intel oneMKL instead of generic OpenBLAS.
- **Verification**: `np.show_config()` displays `libraries = ['mkl_rt']`.

24. Modin (Accelerated Pandas on Intel Xeon)

- **Definition**: A parallel dataframe library that distributes Pandas operations across all available CPU cores using Ray or Dask backends.
- **HPC Mechanism**: Standard Pandas is restricted to 1 CPU core due to the Python GIL. Modin partitions dataframes vertically and horizontally, executing operations concurrently across all 64 cores.

25. daal4py (Python API for oneDAL)

- **Definition**: The low-level Python binding layer directly wrapping oneDAL C++ classes, used under the hood by `sklearnex` for streaming analytics and distributed ML training.

26. Deadlock in Distributed Computing

- **Definition**: A state where two or more processes are permanently blocked because each is waiting for a resource or message that the other holds.
- **Example**: Rank 0 calls blocking `MPI_Recv` waiting for Rank 1, while Rank 1 simultaneously calls blocking `MPI_Recv` waiting for Rank 0. Neither ever reaches `MPI_Send`.

27. Non-Blocking Communication (`MPI_Isend` / `MPI_Irecv`)

- **Definition**: Asynchronous MPI routines that initiate data transfer and return immediately without waiting for the buffer to be transmitted, returning an `MPI_Request` handle.
- **Benefit**: Enables computational overlapping: the CPU performs mathematical operations while the network interface card (NIC) handles PCIe DMA data transfer.

28. Hybrid Programming Model (MPI + OpenMP)

- **Definition**: The industry-standard supercomputing design pattern:
  - **MPI (Inter-node)**: 1 MPI process per CPU socket or NUMA domain communicating across the network.
  - **OpenMP (Intra-node)**: Multiple threads spawned by each MPI rank to saturate local CPU cores via shared memory.

29. Static vs. Dynamic Loop Scheduling (OpenMP)

- **Definition**:
  - `schedule(static, chunk)`: Divides loop iterations into fixed chunks assigned round-robin at compile/start time (ideal for uniform workloads).
  - `schedule(dynamic, chunk)`: Worker threads dynamically request a new chunk from a shared queue when idle (ideal for irregular, non-uniform workloads).

30. Thread Affinity & NUMA Awareness

- **Definition**: Binding threads to specific hardware cores or NUMA memory nodes to ensure thread memory allocations remain local to the core's attached memory controller.
- **Environment Variable**: `export OMP_PROC_BIND=close`, `export OMP_PLACES=cores`.

31. Memory Bound vs. Compute Bound in ML

- **Definition**:
  - **Memory Bound**: Algorithm throughput is limited by DRAM bandwidth (e.g., vector addition, feature normalization, element-wise scaling). Arithmetic intensity is low (< 1 FLOP/byte).
  - **Compute Bound**: Algorithm throughput is limited by processor clock cycles and ALU capacity (e.g., matrix multiplication, deep neural network convolutions). Arithmetic intensity is high (> 10 FLOPs/byte).

32. SIMD Vectorization in Traditional ML

- **Definition**: Utilizing CPU vector registers (AVX2 256-bit or AVX-512 512-bit) to evaluate activation functions, Euclidean distances, and dot products on 8 or 16 floating-point values in parallel.

33. Unpatching Scikit-learn (`unpatch_sklearn`)

- **Definition**: The function to revert `sklearnex` patches back to original stock Scikit-learn behavior:

```
from sklearnex import unpatch_sklearn
unpatch_sklearn()
```

- **Use Case**: Used for benchmarking to run before/after speedup comparisons in the same script without restarting Python.

34. Batch Prediction Throughput (Samples / Second)

- **Definition**: The metric measuring how many inference rows a trained machine learning model can evaluate per second.
- **HPC Formula**: $\text{Throughput} = \frac{\text{Total Test Samples}}{\text{Inference Latency in Seconds}}$.

35. Core Scaling Efficiency

- **Definition**: The percentage of ideal linear performance scaling maintained as CPU core count increases from 1 to N.
- **Formula**: $\text{Scaling Efficiency} = \frac{\text{Speedup on } N \text{ cores}}{N} \times 100\%$.

36. Barrier Synchronization (`MPI_Barrier` / `#pragma omp barrier`)

- **Definition**: A synchronization point where all participating threads or processes must arrive and wait before any thread is permitted to proceed.

Glossary Status: 100% Complete & Verified | GLA University HPC Masterclass Day 2